

WEEK 8 | SCALABLE TRAINING + DEPLOYMENT

Model Packaging & Versioning

From training checkpoint to shipping artifact.

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

SESSION 3 OF 4

WEEK 8 | SCALABLE TRAINING + DEPLOYMENT

The Big Picture — Why This Session Exists

You just finished training a great model. If you hand your raw training checkpoint to another team, there's a good chance they can't even load it.

THINK OF IT LIKE THIS: A chef's messy kitchen notes vs. the polished recipe card in a cookbook. The notes are shorthand only the chef understands. The card is for ANYONE to follow. A training checkpoint is the messy notes; packaging turns it into the recipe card.

1

Checkpoints and artifacts differ

One resumes training; the other is for strangers to run in production.

2

Wrong format = security risk

Some formats can silently execute code when loaded.

3

Without versioning, nobody knows what's running

'Which model is live right now?' needs a fast, confident answer.

Key Terms Before We Start

A quick shared vocabulary — skip ahead if these are all second nature to you.

Artifact — A finished, shareable file (or set of files) meant for others to use.

Serialization — Converting an in-memory object (like model weights) into a saveable file.

Model registry — A catalog tracking every model version and which one is currently 'live.'

Inference — Running a trained model to get a prediction, as opposed to training it.

Versioning — Labeling iterations so you can tell them apart and roll back if needed.

Reproducibility — The ability to recreate the exact same result again later.

Checkpoint vs Artifact — Different Jobs

Checkpoint

Save-and-resume during training.
Weights + optimizer + RNG + step.
Pickle (torch.save) is fine.
Ephemeral — deleted after training.

Artifact

Shipped to downstream consumers.
Weights only.
Framework-independent, ideally.
Immutable — versioned forever.

THE HANDOFF CONTRACT: at the end of training, CONVERT a checkpoint into an artifact. Strip optimizer state. Pick a portable format. Stamp a version. Register it.

Why Not Just Reuse the Training Checkpoint?

It seems tempting to ship the same file used during training. Here's what breaks.

What it has	Why it's a problem
Optimizer state (often 4x model size)	Useless for inference, bloats the file 4-5x
A dependency on your exact class code	The inference team's code may differ — loading fails
No documentation of limitations	Nobody downstream knows what's safe to use it for
No security guarantees on the format	Some formats execute code the moment they load

safetensors — Ditch Pickle for Weights

THINK OF IT LIKE THIS: A pickle file is a mystery box that might contain a trap. safetensors is a clear, sealed envelope — you can see exactly what's inside (just numbers) before ever 'opening' it.

Why pickle is risky

Arbitrary code execution on load. Slow for big models. Ties the file to your exact Python classes.

safetensors (HF, 2022)

Pure numeric format — no executable code. Zero-copy fast loading. Framework-independent.

```
from safetensors.torch import save_file, load_file
save_file(model.state_dict(), 'model.safetensors', metadata={'version': '1.2.3'})
# SAVE
state = load_file('model.safetensors'); model.load_state_dict(state)
# LOAD — safe
```

Three Runtime Formats — Which Fits

TorchScript

No Python runtime needed; mobile-friendly.
2-3x faster than eager.
Being phased out for new work.

torch.compile

Modern PyTorch-native default.
Up to 2x speed, model-dependent.
Still PyTorch 2.x runtime.

ONNX

Truly universal — TensorRT, OpenVINO, CoreML.
Speed depends on the runtime.
Not every op exports cleanly.

DECISION GUIDE: staying in PyTorch, want speed -> torch.compile. Need cross-framework -> ONNX. Maintaining old code -> TorchScript.

The Packaging Recipe (1/2) — Prepare the Weights

A six-step recipe. Steps 1-3 here; steps 4-6 next slide.

```
# 1. Load your finished checkpoint and rebuild the model
state = load_checkpoint(ckpt_path); model = build_model(state['config'])
model.load_state_dict(state['model']); model.eval()

# 2. Save JUST the weights in safetensors — your primary shipping artifact
save_file(model.state_dict(), 'artifacts/model.safetensors', metadata={'version':
'1.2.3'})

# 3. Export a portable graph, for TensorRT / ONNX Runtime teams
torch.onnx.export(model, example, 'artifacts/model.onnx', opset_version=17)
```

Now you have two files that anyone, in any framework, can load — zero dependency on your training script.

The Packaging Recipe (2/2) — Document and Register

Steps 4-6: the parts people skip under deadline pressure, and the parts that matter most six months later.

```
# 4. Write a model card — the passport that travels with the weights
with open('artifacts/MODEL_CARD.md', 'w') as f: f.write(model_card_markdown)

# 5. Bundle config + tokenizer + preprocessing as ONE unit
save_bundle('artifacts/', config=cfg, tokenizer=tok, preproc=preproc)

# 6. Register with a model registry so it's discoverable
mlflow.register_model('runs:/<run_id>/artifacts', 'my-llm')
```

Steps 1-3 make the model LOADABLE. Steps 4-6 make it USABLE and TRUSTWORTHY — a stranger needs to know what it's for and where to find it. Skipping these is the biggest cause of 'which model is this?' confusion later.

Model Registry — Why You Need One

THINK OF IT LIKE THIS: A model registry is a library's card catalog. Without it, models pile up on random shelves. With it, every model has a catalog entry: edition, location, status.

None Just registered, not approved yet.

Staging Under evaluation — shadow traffic, A/B.

Production Live, serving real traffic.

Archived Rolled back, kept for audit.

```
client.transition_model_version_stage(name='my-llm', version=3, stage='Production')
```

Versioning Discipline

DON'T	DO
my-llm:latest — unknown contents	Semver tags — a compatibility signal
model_v3_FINAL_v2 — filename versioning	SHA-tagged files — a fixed identifier
Mutating registry tags, losing history	Registry aliases — always resolve to a known version
No model card, no provenance	A model card next to the weights

SAME RULE AS DOCKER (Week 2): tags are convenient pointers, SHA digests are truth. Production references digests.

Anatomy of a Model Card

The artifact's passport — what a stranger six months from now reads before trusting the weights.

```
## my-llm v1.2.3
- Base model: llama3-8b    Trained on: internal-corpus-v4
- Eval scores: MMLU 0.782 (baseline: 0.87 internal-eval)
- Known limitations: degraded on non-English inputs
- Intended use: internal support-ticket triage only
- Trained by: @username    Approved by: @reviewer
```

- Eval scores let a reviewer judge candidacy for promotion at a glance
- Known limitations prevent the #1 cause of incidents: use outside validated scope
- Trained-by/approved-by makes an audit answerable in minutes

Common Mistakes & How to Spot Them

Mistake	How it shows up	Fix
Shipping the raw checkpoint	Inference team can't load it, or drags in optimizer state	Convert to a safetensors artifact
Using pickle for shared files	A security scan flags it, or a corrupt file runs code	Use safetensors instead
Using 'latest' tags in prod	Nobody knows which version served a request	Use SHA digests or pinned aliases
No model card	Model applied outside its validated use case	Always write limitations + intended use

Quick Recap — Terms and Concepts

Checkpoint vs artifact — Checkpoint = resume training; artifact = ship to others

safetensors — Safe, fast, framework-independent format — replaces pickle

Packaging recipe — safetensors + ONNX + model card + config bundle + registry

Model registry — A catalog: None -> Staging -> Production -> Archived

Versioning discipline — Semver for humans, SHA digests for production truth